

Contents

Tarea — Del modelo entrenado a la API.....	2
1. Introducción	2
2. Metodología.....	2
3. Matriz del paso a paso	3
Desarrollo de la tarea.....	4
Paso 1. Ejecutar y leer el notebook	4
Paso 2. Construir el bundle	5
Paso 3. Guardar el bundle.....	7
Paso 4. Verificar el funcionamiento del bundle.....	7
Paso 5. Construir el contrato en esquemas.py	8
Paso 6. Construir la API en main.py	10
Paso 7. Definir la salida de /predecir	11
Paso 8. Levantar y probar el servicio desde /docs.....	12
Paso 9. Preparar la entrega final.....	14

Tarea — Del modelo entrenado a la API

1. Introducción

En esta tarea se parte de un modelo de Machine Learning previamente entrenado para predecir la cancelación de clientes. Se dispone del archivo `clientes.csv`, el notebook `entrenar_modelo.ipynb` y el archivo `requirements.txt`. El trabajo no consiste en desarrollar nuevamente el modelo, sino en tomar el modelo ya entrenado y prepararlo para ser utilizado como un servicio mediante una API.

Para ello, se construirá un **bundle** que agrupe el modelo entrenado y la información necesaria para utilizarlo posteriormente en producción. Este artefacto será serializado utilizando `joblib` y posteriormente se verificará que, al ser cargado nuevamente, mantenga exactamente el mismo comportamiento predictivo que el modelo original.

Posteriormente se construirá el contrato de entrada de datos utilizando **Pydantic** y una API mediante **FastAPI**, con un endpoint para verificar el estado del servicio y otro para realizar predicciones de cancelación. La API deberá recibir los datos de un cliente, calcular su probabilidad de cancelación y devolver la predicción, el nivel de riesgo y el umbral utilizado.

Finalmente, el servicio será ejecutado y probado desde la documentación interactiva `/docs` de FastAPI utilizando casos de riesgo alto y riesgo bajo, documentando mediante capturas de pantalla las principales etapas realizadas.

2. Metodología

La metodología utilizada seguirá directamente las instrucciones establecidas para la tarea y se desarrollará de forma secuencial, partiendo del modelo ya entrenado hasta obtener un servicio de predicción funcional.

El proceso comenzará con la ejecución y revisión completa del notebook entregado, con el objetivo de identificar las variables, configuraciones y resultados que deberán conservarse para producción. A partir de estas variables se construirá un diccionario denominado **bundle**, evitando ingresar manualmente valores que ya existan dentro del notebook.

El bundle será serializado en el archivo `modelo_churn.joblib` utilizando `joblib.dump()`. Posteriormente se realizará una validación mediante la recarga del artefacto y la repetición de la predicción sobre el mismo cliente utilizado en el notebook. Esta verificación permitirá confirmar que el modelo serializado conserva exactamente su capacidad de inferencia.

Una vez validado el artefacto, se construirá el contrato de entrada en `esquemas.py` utilizando Pydantic, incluyendo los ocho campos requeridos, sus rangos válidos y las categorías permitidas.

Después se desarrollará `main.py` mediante FastAPI, incorporando los dos endpoints solicitados: `GET /` y `POST /predecir`.

El endpoint de predicción utilizará el modelo almacenado en el bundle para calcular la probabilidad de cancelación y generará como salida la predicción correspondiente según el umbral, el nivel de riesgo y el umbral utilizado.

Finalmente, la API será levantada y probada mediante `/docs` con un cliente de riesgo alto y otro de riesgo bajo. Las evidencias de las etapas relevantes serán documentadas mediante capturas de pantalla para incorporarlas al documento final de entrega.

3. Matriz del paso a paso

Paso	Actividad solicitada	Acción a realizar	Resultado / evidencia esperada
1	Ejecutar y leer el notebook	Ejecutar completamente <code>entrenar_modelo.ipynb</code> y revisar las variables, modelo, métricas y clientes utilizados en las pruebas.	Notebook ejecutado correctamente y comprensión de las variables disponibles.
2	Construir el bundle	Crear un diccionario utilizando las variables existentes del notebook para almacenar el modelo, columnas de entrada, numéricas, categóricas, valores válidos de <code>plan</code> y <code>metodo_pago</code> , umbral, métricas, fecha de entrenamiento, filas utilizadas y versiones de librerías.	Diccionario <code>bundle</code> construido y mostrado correctamente en el notebook.
3	Guardar el bundle	Serializar el diccionario mediante <code>joblib.dump()</code> con el nombre <code>modelo_churn.joblib</code> .	Archivo <code>modelo_churn.joblib</code> generado correctamente.
4	Verificar el bundle	Cargar nuevamente <code>modelo_churn.joblib</code> y realizar la predicción sobre el mismo cliente utilizado con el modelo original.	La probabilidad obtenida debe ser idéntica a la predicción original.
5	Construir el contrato	Crear <code>esquemas.py</code> con la clase <code>Cliente</code> , incluyendo los ocho campos, los rangos válidos y los valores permitidos de <code>plan</code> y <code>metodo_pago</code> .	Archivo <code>esquemas.py</code> con el contrato de entrada definido.
6	Construir la API	Crear <code>main.py</code> utilizando FastAPI e implementar los endpoints <code>GET /</code> y <code>POST /predecir</code> .	API construida y capaz de cargar el bundle y recibir solicitudes.

Paso	Actividad solicitada	Acción a realizar	Resultado / evidencia esperada
7	Definir la salida de predicción	Configurar <code>/predecir</code> para devolver probabilidad de cancelación, predicción <code>cancela o sigue</code> según el umbral, nivel de riesgo alto, medio o bajo, y el umbral utilizado.	Respuesta de la API con los cuatro elementos solicitados.
8	Levantar y probar el servicio	Ejecutar la API y realizar pruebas desde <code>/docs</code> con un cliente de riesgo alto y otro de riesgo bajo.	Predicciones correctas y capturas de ambas pruebas.
9	Preparar la entrega	Entregar el notebook completado, los archivos <code>.py</code> solicitados y el documento con la explicación del procedimiento y las capturas realizadas.	Conjunto final de archivos y documento explicativo.

Desarrollo de la tarea

Paso 1. Ejecutar y leer el notebook

Se ejecutó completamente el notebook `entrenar_modelo.ipynb` suministrado para la tarea, con el objetivo de revisar el proceso de entrenamiento, identificar las variables disponibles y comprobar el comportamiento del modelo antes de iniciar su preparación para producción.

El notebook trabaja con un conjunto de datos de **6.000 clientes** y utiliza **8 variables de entrada** para predecir la cancelación de clientes. El modelo ya se encontraba construido mediante un `Pipeline` que integra el preprocesamiento de las variables y un modelo de **Regresión Logística**, por lo que no fue necesario realizar modificaciones al proceso de entrenamiento.

Durante la revisión se identificaron las variables necesarias para las siguientes etapas de la tarea, entre ellas las columnas de entrada, las variables numéricas y categóricas, las métricas del modelo, las versiones de las librerías y los clientes utilizados para realizar pruebas de predicción.

Los principales resultados obtenidos por el modelo fueron los siguientes:

Métrica Resultado

Accuracy 0.8527

Precision 0.8215

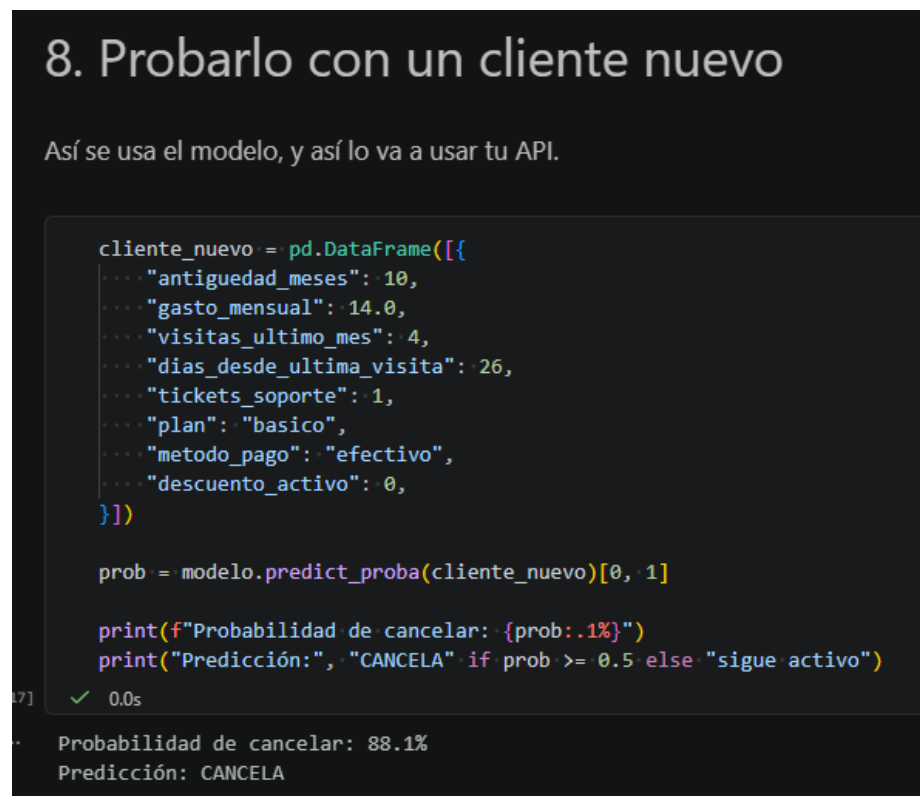
Recall 0.6942

Métrica Resultado

F1-score 0.7525

ROC-AUC 0.9055

Como parte de la verificación del modelo original, se utilizó el cliente de prueba incluido en el notebook. El modelo obtuvo una **probabilidad de cancelación aproximada del 88.1 %**, clasificando al cliente como un caso de cancelación. Este resultado se utilizará posteriormente como referencia para comprobar que el modelo conserva exactamente su comportamiento después de ser guardado y cargado desde el archivo `modelo_churn.joblib`.



```
8. Probarlo con un cliente nuevo

Así se usa el modelo, y así lo va a usar tu API.

cliente_nuevo = pd.DataFrame([
    {"antiguedad_meses": 10,
     "gasto_mensual": 14.0,
     "visitas_ultimo_mes": 4,
     "dias_desde_ultima_visita": 26,
     "tickets_soporte": 1,
     "plan": "basico",
     "metodo_pago": "efectivo",
     "descuento_activo": 0,
    }])

prob = modelo.predict_proba(cliente_nuevo)[0, 1]

print(f"Probabilidad de cancelar: {prob:.1%}")
print("Predicción:", "CANCELA" if prob >= 0.5 else "sigue activo")

17] ✓ 0.0s

Probabilidad de cancelar: 88.1%
Predicción: CANCELA
```

Figura 1. Ejecución y verificación del modelo original.

La captura muestra la ejecución del cliente de prueba incluido en el notebook y el resultado obtenido por el modelo, con una probabilidad de cancelación aproximada del 88.1 % y una predicción de cancelación.

Paso 2. Construir el bundle

A partir de las variables disponibles en el notebook se construyó un diccionario denominado `bundle`, encargado de agrupar el modelo entrenado y la información necesaria para su posterior utilización en producción.

El bundle incluye el modelo, las columnas de entrada, las variables numéricas y categóricas, los valores válidos de `plan` y `metodo_pago`, el umbral de clasificación, las métricas del modelo, la fecha de entrenamiento, la cantidad de filas utilizadas y las versiones de las principales librerías.

Los valores fueron obtenidos directamente de las variables y estructuras existentes en el notebook, evitando escribir manualmente información que ya se encontraba disponible. Como comprobación adicional, se imprimieron las claves almacenadas en el diccionario, verificando que todos los elementos solicitados por la tarea estuvieran presentes.

```
# TAREA- Parte 1: armá el diccionario 'bundle'

UMBRAL = 0.4

bundle = {
    "modelo": modelo,
    "columnas": COLUMNAS,
    "columnas_numericas": COLUMNAS_NUMERICAS,
    "columnas_categoricas": COLUMNAS_CATEGORICAS,
    "planes_validos": sorted(clientes["plan"].unique().tolist()),
    "metodos_pago_validos": sorted(clientes["metodo_pago"].unique().tolist()),
    "umbral": UMBRAL,
    "metricas": {
        "accuracy": float(exactitud),
        "precision": float(precision),
        "recall": float(sensibilidad),
        "f1": float(f1),
        "roc_auc": float(auc),
    },
    "fecha_entrenamiento": pd.Timestamp.now().strftime("%Y-%m-%d %H:%M:%S"),
    "filas_usadas": len(X_entrena),
    "versiones": {
        "pandas": pd.__version__,
        "numpy": np.__version__,
        "scikit_learn": sklearn.__version__,
        "joblib": joblib.__version__,
    },
}

print("Claves del bundle:")
for clave in bundle.keys():
    print("-", clave)
```

✓ 0.0s

```
Claves del bundle:
- modelo
- columnas
- columnas_numericas
- columnas_categoricas
- planes_validos
- metodos_pago_validos
- umbral
- metricas
- fecha_entrenamiento
- filas_usadas
- versiones
```

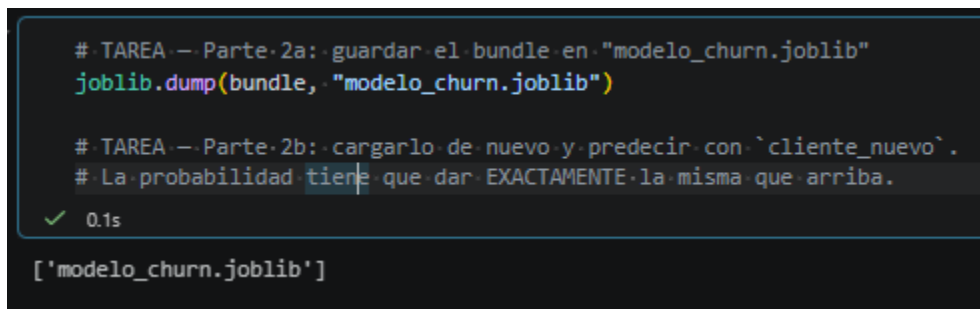
Figura 2. Construcción y verificación del bundle.

La captura muestra la creación del diccionario `bundle` y la comprobación de las claves almacenadas, confirmando la inclusión de los elementos requeridos para conservar el modelo y su información asociada.

Paso 3. Guardar el bundle

Una vez construido y verificado el contenido del `bundle`, se procedió a guardarlo mediante la función `joblib.dump()`, utilizando el nombre de archivo `modelo_churn.joblib`.

Este proceso permite serializar el diccionario y conservar en un archivo el modelo entrenado junto con la información incorporada previamente en el bundle. La ejecución generó correctamente el archivo `modelo_churn.joblib`, el cual será utilizado posteriormente para comprobar que el modelo puede cargarse nuevamente y mantener el mismo resultado de predicción.



```
# TAREA - Parte 2a: guardar el bundle en "modelo_churn.joblib"
joblib.dump(bundle, "modelo_churn.joblib")

# TAREA - Parte 2b: cargarlo de nuevo y predecir con `cliente_nuevo`.
# La probabilidad tiene que dar EXACTAMENTE la misma que arriba.

✓ 0.1s

['modelo_churn.joblib']
```

Figura 3. Guardado del bundle en `modelo_churn.joblib`.

La captura muestra la ejecución de `joblib.dump()` y la confirmación de la generación del archivo `modelo_churn.joblib`.

Paso 4. Verificar el funcionamiento del bundle

Después de guardar el bundle, se cargó nuevamente el archivo `modelo_churn.joblib` utilizando `joblib.load()`. Posteriormente, se realizó una nueva predicción utilizando el mismo `cliente_nuevo` empleado previamente con el modelo original.

La probabilidad obtenida con el modelo original fue **0.881333**, mientras que el modelo cargado desde el archivo `modelo_churn.joblib` produjo igualmente una probabilidad de **0.881333**. La comparación entre ambos valores dio como resultado `True`, confirmando que las probabilidades son exactamente iguales.

Con esta comprobación se verificó que el proceso de guardado y carga del bundle conserva correctamente el modelo entrenado y su capacidad de predicción.

```
# TAREA — Parte 2a: guardar el bundle en "modelo_churn.joblib"
joblib.dump(bundle, "modelo_churn.joblib")

# TAREA — Parte 2b: cargarlo de nuevo y predecir con `cliente_nuevo`.
# La probabilidad tiene que dar EXACTAMENTE la misma que arriba.

bundle_cargado = joblib.load("modelo_churn.joblib")

prob_recargada = bundle_cargado["modelo"].predict_proba(cliente_nuevo)[0, 1]

print(f"Probabilidad modelo original: {prob:.6f}")
print(f"Probabilidad modelo recargado: {prob_recargada:.6f}")
print("¿Son exactamente iguales?:", prob == prob_recargada)

✓ 0.1s

Probabilidad modelo original: 0.881333
Probabilidad modelo recargado: 0.881333
¿Son exactamente iguales?: True
```

Figura 4. Verificación del modelo después de cargar el bundle.

La captura muestra la comparación entre la probabilidad obtenida con el modelo original y la obtenida después de cargar `modelo_churn.joblib`, confirmando que ambos resultados son exactamente iguales.

Paso 5. Construir el contrato en `esquemas.py`

Se creó el archivo `esquemas.py` para definir el contrato de entrada de datos que utilizará posteriormente la API. En este archivo se implementó la clase `Cliente` mediante `BaseModel` de Pydantic, incorporando los ocho campos utilizados por el modelo.

Para las variables numéricas se establecieron sus respectivos rangos válidos mediante `Field`, utilizando los límites `ge` (mayor o igual) y `le` (menor o igual). Para las variables categóricas `plan` y `metodo_pago` se utilizó `Literal`, restringiendo los datos de entrada únicamente a los valores permitidos.

De esta manera, el contrato permite validar que la información recibida de cada cliente tenga la estructura, tipos, rangos y categorías requeridas antes de ser enviada al modelo para realizar una predicción.

```

Tarea > esquemas.py > Cliente
1  from typing import Literal
2
3  from pydantic import BaseModel, Field
4
5
6  class Cliente(BaseModel):
7      """antigüedad_meses: int = Field(
8          """ge=1,
9          """le=60,
10         """description="Antigüedad del cliente en meses"
11         """
12
13         """gasto_mensual: float = Field(
14             """ge=5.0,
15             """le=58.39,
16             """description="Gasto mensual del cliente"
17             """
18
19         """visitas_ultimo_mes: int = Field(
20             """ge=0,
21             """le=29,
22             """description="Cantidad de visitas realizadas durante el último mes"
23             """
24
25         """dias_desde_ultima_visita: int = Field(
26             """ge=0,
27             """le=90,
28             """description="Días transcurridos desde la última visita"
29             """
30
31         """tickets_soporte: int = Field(
32             """ge=0,
33             """le=5,
34             """description="Cantidad de tickets de soporte"
35             """
36
37         """plan: Literal["basico", "estandar", "premium"] = Field(
38             """description="Plan contratado por el cliente"
39             """
40
41         """metodo_pago: Literal["efectivo", "tarjeta", "transferencia"] = Field(
42             """description="Método de pago utilizado por el cliente"
43             """
44
45         """descuento_activo: int = Field(
46             """ge=0,
47             """le=1,
48             """description="Indica si el cliente tiene un descuento activo: 0 = No, 1 = Si"
49             """

```

Figura 5. Definición del contrato de entrada en `esquemas.py`.

La captura muestra la clase `Cliente` con los ocho campos requeridos, incluyendo las validaciones correspondientes para las variables numéricas y los valores permitidos para las variables categóricas.

Paso 6. Construir la API en main.py

Se creó el archivo `main.py` utilizando FastAPI para implementar el servicio de predicción del modelo de cancelación de clientes. La API fue configurada para cargar el archivo `modelo_churn.joblib` al iniciar el servicio y mantener el bundle disponible para realizar las predicciones.

Se implementaron los dos endpoints solicitados. El endpoint `GET /` permite consultar el estado del servicio y verificar si el modelo se encuentra cargado, mientras que el endpoint `POST /predecir` recibe los datos de un cliente utilizando el contrato definido previamente en `esquemas.py`.

Dentro del endpoint de predicción, los datos recibidos se convierten en un diccionario y posteriormente en un DataFrame, respetando el orden de las columnas almacenadas en el bundle. Finalmente, el modelo calcula mediante `predict_proba()` la probabilidad de cancelación del cliente.

```
Tarea > main.py > predecir
1  from contextlib import asynccontextmanager
2
3  import joblib
4  import pandas as pd
5  from fastapi import FastAPI, HTTPException
6
7  from esquemas import Cliente
8
9
10 NOMBRE_BUNDLE = "modelo_churn.joblib"
11
12 estado_servicio = {"bundle": None}
13
14
15 @asynccontextmanager
16 async def lifespan(app: FastAPI):
17     estado_servicio["bundle"] = joblib.load(NOMBRE_BUNDLE)
18     print("Bundle cargado correctamente")
19     yield
20     estado_servicio["bundle"] = None
21
22
23 app = FastAPI(
24     title="API de predicción de cancelación de clientes",
25     description="Recibe los datos de un cliente y predice su probabilidad de cancelación.",
26     version="1.0.0",
27     lifespan=lifespan
28 )
29
30
31 @app.get("/")
32 def estado():
33     return {
34         "servicio": "API de predicción de cancelación de clientes",
35         "modelo_cargado": estado_servicio["bundle"] is not None
36     }
37
38
39 @app.post("/predecir")
40 def predecir(cliente: Cliente):
41     bundle = estado_servicio["bundle"]
42
43     if bundle is None:
44         raise HTTPException(
45             status_code=503,
46             detail="Modelo no cargado"
47         )
48
49     fila = cliente.model_dump()
50
51     X_nuevo = pd.DataFrame([fila])[bundle["columnas"]]
52
53     probabilidad = bundle["modelo"].predict_proba(X_nuevo)[0, 1]
54
55     return {
56         "probabilidad": round(float(probabilidad), 4)
57     }
```

Figura 6. Construcción de la API en main.py.

La captura muestra la configuración de FastAPI, la carga del bundle y la implementación de los endpoints `GET /` y `POST /predecir`.

Paso 7. Definir la salida de /predecir

Se completó el endpoint `POST /predecir` para devolver la información solicitada por la tarea. Primero se obtiene la probabilidad de cancelación mediante `predict_proba()` y posteriormente se recupera el umbral almacenado en el bundle.

La predicción se determina comparando la probabilidad con dicho umbral: cuando la probabilidad es mayor o igual al umbral, el resultado es `cancela`; en caso contrario, el resultado es `sigue`.

Adicionalmente, se definió el nivel de riesgo en tres categorías: bajo, medio y alto. Se consideró riesgo bajo cuando la probabilidad se encuentra por debajo del umbral, riesgo medio cuando se encuentra entre el umbral y 0.7, y riesgo alto cuando la probabilidad es igual o superior a 0.7.

Finalmente, la respuesta del endpoint incluye la probabilidad de cancelación, la predicción, el nivel de riesgo y el umbral utilizado.

```
38
39 @app.post("/predecir")
40 def predecir(cliente: Cliente):
41     bundle = estado_servicio["bundle"]
42
43     if bundle is None:
44         raise HTTPException(
45             status_code=503,
46             detail="Modelo no cargado"
47         )
48
49     fila = cliente.model_dump()
50
51     X_nuevo = pd.DataFrame([fila])[bundle["columnas"]]
52
53     probabilidad = bundle["modelo"].predict_proba(X_nuevo)[0, 1]
54
55     umbral = bundle["umbral"]
56
57     prediccion = "cancela" if probabilidad >= umbral else "sigue"
58
59     if probabilidad >= 0.7:
60         riesgo = "alto"
61     elif probabilidad >= umbral:
62         riesgo = "medio"
63     else:
64         riesgo = "bajo"
65
66     return {
67         "probabilidad_cancelacion": round(float(probabilidad), 4),
68         "prediccion": prediccion,
69         "nivel_riesgo": riesgo,
70         "umbral_usado": umbral
71     }
```

Figura 7. Configuración de la salida del endpoint `/predecir`.

La captura muestra la lógica utilizada para calcular la predicción, clasificar el nivel de riesgo y devolver los cuatro elementos requeridos por la tarea.

Paso 8. Levantar y probar el servicio desde /docs

Una vez construida la API, se levantó el servicio mediante Uvicorn y se accedió a la documentación interactiva `/docs` generada por FastAPI. Desde esta interfaz se verificó la disponibilidad de los endpoints `GET /` y `POST /predecir`.

Posteriormente se realizaron las dos pruebas solicitadas utilizando el endpoint `POST /predecir`. Para el cliente de riesgo alto se obtuvo una probabilidad de cancelación de **0.8813**, generando la predicción `cancela` y un nivel de riesgo `alto`, con un umbral de 0.4.

Para el cliente de riesgo bajo se obtuvo una probabilidad de cancelación de **0.0124**, generando la predicción `sigue` y un nivel de riesgo `bajo`, utilizando igualmente el umbral de 0.4.

En ambos casos el servicio respondió correctamente con código HTTP 200, comprobando el funcionamiento de la API y la generación de las salidas requeridas.

API de predicción de cancelación de clientes 1.0.0 OAS 3.1

[/openapi.json](#)

Recibe los datos de un cliente y predice su probabilidad de cancelación.

The screenshot displays the FastAPI interactive API documentation interface. At the top, it shows the API title "API de predicción de cancelación de clientes" with version "1.0.0" and "OAS 3.1". Below the title, there's a description: "Recibe los datos de un cliente y predice su probabilidad de cancelación." and a link to "/openapi.json".

The interface is divided into two main sections. The top section is for the "default" environment, showing the "GET / Estado" endpoint. It has a "Parameters" tab with "No parameters" and a "Responses" tab showing a "200 Successful Response" with a media type of "application/json".

The bottom section is for the "POST /predecir" endpoint. It has a "Parameters" tab with "No parameters" and a "Request body" tab. The "Request body" tab is active, showing a "required" field with a dropdown menu set to "application/json". Below the dropdown, there's a "Edit Value" section with a "Schema" tab. The schema is a JSON object with the following fields: "antiguedad_meses" (10), "gasto_mensual" (14.0), "visitas_ultimo_mes" (4), "dias_desde_ultima_visita" (26), "tickets_susporte" (1), "plan" ("basico"), "metodo_pago" ("efectivo"), and "descuento_activa" (0).



Figura 8. Prueba de cliente con riesgo alto desde /docs.

La captura muestra la respuesta obtenida para un cliente con probabilidad de cancelación de 0.8813, clasificado como *cancela* y con nivel de riesgo *alto*.



Figura 9. Prueba de cliente con riesgo bajo desde /docs.

La captura muestra la respuesta obtenida para un cliente con probabilidad de cancelación de 0.0124, clasificado como *sigue* y con nivel de riesgo *bajo*.

Paso 9. Preparar la entrega final

Una vez finalizada la construcción y validación del servicio, se organizaron los archivos correspondientes para realizar la entrega de la tarea.

El notebook `entrenar_modelo.ipynb` fue completado con la construcción del bundle, su serialización mediante `joblib` y la comprobación de que el modelo cargado conserva exactamente la misma probabilidad de predicción que el modelo original.

También se incorporaron los archivos `esquemas.py`, que contiene el contrato de entrada de datos mediante Pydantic, y `main.py`, que contiene la implementación de la API mediante FastAPI. Adicionalmente, se conservó el archivo `modelo_churn.joblib`, generado durante la serialización del bundle y utilizado posteriormente por el servicio para realizar las predicciones.

Finalmente, se preparó el presente documento con la descripción del procedimiento realizado y las capturas correspondientes a las principales etapas de desarrollo y comprobación de la tarea.